CHAPTER 28

Digital Wallet

Transfer balances between wallets at 1M TPS with strong consistency and full auditability.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | API / data model | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design a **digital wallet** (PayPal balance / Venmo / Alipay) that stores user balances and supports **balance transfers** between wallets. The challenge: provide **correct, exactly-once, atomic transfers at very high throughput** (1 M TPS) with auditability, evolving from a simple in-memory design through a relational DB and distributed transactions to **event sourcing**.

Why interviewers ask it: A deep-dive in distributed transactions: 2PC vs TC/C vs Saga vs event sourcing, plus reproducibility.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Features?	Wallet-to-wallet balance transfer.
TPS?	1,000,000 TPS.
Correctness?	Transactional guarantees; **reproducibility** (reconstruct state from history).
Reliability?	99.99% availability.
Currency?	Single currency (extensible).

3. Requirements

Functional

- Transfer amount from wallet A to B atomically.
- Query balance.
- Transaction history / audit.

Non-functional

- **Consistency / atomicity:** no lost or created money.
- **Reproducibility:** rebuild any past state; audit.
- High availability and durability.
- 1 M TPS throughput.

4. Back-of-the-envelope estimation

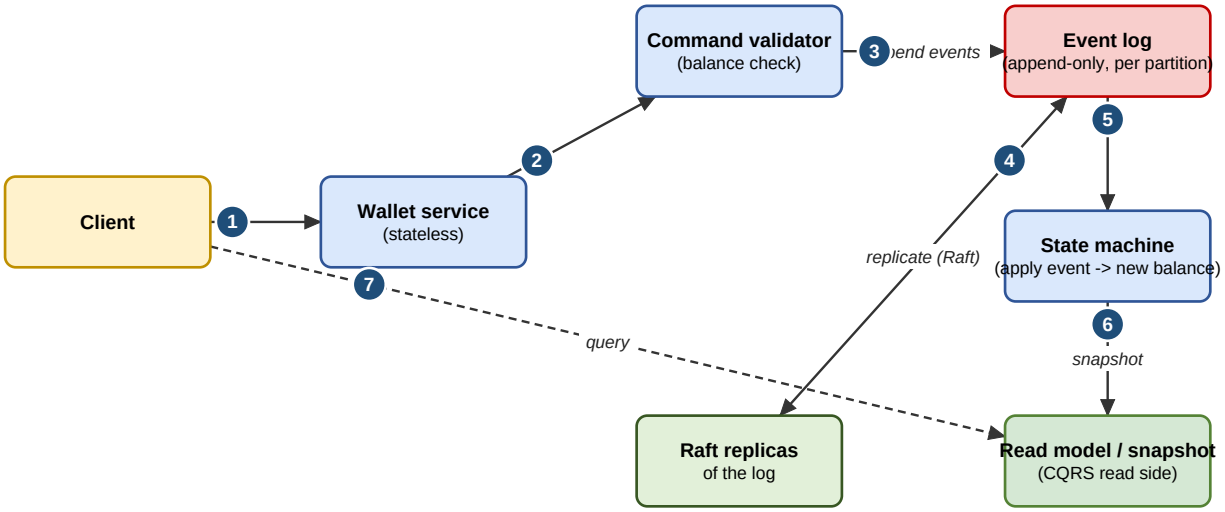
- 1 M TPS x 2 balance updates = 2 M updates/s. A single relational DB node handles about 1 K-10 K TPS -> need **at least 100-1,000 nodes** (about 1,000 TPS/node x 1,000 nodes) -> data partitioning is essential.
- A transfer touches two accounts that may live on different partitions -> cross-partition transaction unless co-located.

5. API / data model

```
POST /v1/wallet/balance_transfer  { transaction_id (idempotency), from_account, to_account, amount, currency }
Tables per partition: account(account_id, balance)  ;  event log: (seq, transfer_id, from, to, amount)
```

6. High-level design

Evolution: in-memory -> relational + 2PC/Saga -> event sourcing (CQRS)



Step	What happens
1	The client submits a transfer: `A -> B, $1` with a unique transfer ID.
2	The stateless wallet service forwards it to the **command validator**, which checks the balance of A (from the current state), amount validity and idempotency.
3	If valid, it **appends events** `A: -$1` and `B: +$1` to the **event log**, ordered within a partition (for example by transfer ID or account).
4	The log is **replicated with Raft** to the other replicas. The command is acknowledged only after a majority has it.
5	The **state machine** reads events in order and applies them deterministically to produce new balances.
6	Results are written to the **read model / snapshot** (CQRS read side), so the log doesn't need replaying for every read.
7	**Query:** balance reads are served from the read model; because it is derived, it may lag slightly, and the client can poll or be notified.

7. Deep dive

Option 1: In-memory sharded store (Redis)

- Partition accounts with consistent hashing (`account_id`) across Redis nodes; a ZooKeeper-like config holds the mapping. Fast, but a transfer across two nodes is **not atomic** (two separate updates) and Redis lacks multi-node transactions/durability -> fails correctness.

Option 2: Relational DB with distributed transactions

- Shard relational DBs by account. Same-shard transfers: single local ACID transaction. Cross-shard: need a protocol.
- **Two-phase commit (2PC):** coordinator -> prepare (lock rows) -> commit/abort. Simple semantics but **blocking**, coordinator is a SPOF, and low performance (locks held across network round trips).
- **TC/C (Try-Confirm/Cancel):** Try phase reserves (A: deduct, B: no-op), Confirm applies (B: add), Cancel compensates (refund A). Each phase is a **local transaction**; no long locks; the **coordinator persists phase status** in a table so it can resume after failure. Needs idempotent, **commutative-ordering-safe** operations; careful with out-of-order Cancel/Try (empty rollback, hanging operations).
- **Saga:** a linear chain of local transactions with compensating actions; orchestrated (central coordinator) or choreographed (events). Eventual consistency; operations in a saga are not isolated, so intermediate states are visible.
- Problem: none of these give an easy **audit trail** or **reproducibility**, and bugs corrupt the state with no ability to rebuild it.

Option 3: Event sourcing (the preferred design)

- Store **immutable events** (facts: `A -> B, $1`) in an append-only log instead of mutable balances; the current balance is a **fold** over the events. Four concepts: **command** (intent, can be rejected), **event** (validated fact), **state** (balances), **state machine** (validates command -> generates events -> applies events to the state).
- Flow: command queue -> **command validator** (balance sufficient?) -> event list -> state machine applies -> state updated. Reproducibility: replay events from the start (or from a snapshot) to rebuild or audit any point in time; bugs can be fixed by replaying.
- **CQRS:** write path appends events; read path builds **read-only state views** (balance, history) asynchronously for queries - decouples reads and writes; push model for updates.

Making event sourcing fast and reliable

- **Local file-based storage:** commands/events in an append-only **mmap'ed file** on the local node (no network hop per event) with batched fsync; state in **RocksDB / SQLite** locally; **snapshots** periodically to cut replay time.
- **Reliability through consensus:** replicate the event log across nodes with **Raft** (leader writes, followers replicate, majority commit). A node crash is recovered by promoting a follower with an identical log; hence, high durability and availability (target no data loss).
- **Partitioning:** each Raft group (partition) owns a subset of accounts; many groups in parallel produce the 1 M TPS; cross-partition transfers use TC/C or Saga on top, where each leg is an event in its own partition's log.

Pull vs push for read side

- Pull: external client polls (inefficient, delayed). Push: read-only state machine pushes updates to the reverse proxy / users (reactive style, timely), with the **reverse proxy** providing synchronous-looking APIs over the async event flow.

Cross-partition transfer in the final design

- Coordinator creates a **distributed transaction** record: Partition A (try: deduct) -> Partition B (confirm: add) with durable status; on failure Cancel returns funds. Idempotent by `transaction_id`.

8. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Atomic cross-account	2PC / TC/C / Saga / co-located shards	TC/C or Saga for performance; 2PC only for rare, small-scale needs.
State storage	Mutable rows / event sourcing	Event sourcing: audit + replay + easy debug; more complexity.
Replication	Async / consensus (Raft)	Raft for no-loss durability at the cost of latency (milliseconds).
Consistency vs performance	Strong everywhere / strong per partition + eventual views	Strong per partition; reads via CQRS views.

9. Scaling, reliability, failure modes

- Add partitions (Raft groups) horizontally; rebalance by moving accounts via snapshot + log tail.
- Hot accounts (merchants) throttled/sub-accounted.
- Regular snapshots; verify balances by replaying logs in the background (continuous audit).

10. Likely follow-up questions

- What if a TC/C Cancel arrives before Try? (empty rollback)
- How do you prevent negative balances under concurrency? (single-writer per account partition)
- How do you migrate accounts between partitions?
- How do you support multi-currency transfers?

11. Common mistakes

- Using Redis as the system of record for money.
- Presenting 2PC as free: ignoring its blocking and SPOF behaviour.
- No idempotency key on transfers.

12. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"Scope: balance transfers between wallets only. The target is 1,000,000 TPS, so a single relational DB is not enough, and correctness is required: no money created or lost, strict serializability per account, and a reproducible audit trail."
2. High-level design	10 min	"I'd show evolution: an in-memory key-value store like Redis is fast but not durable or transactional across nodes. Then a sharded relational DB with two-phase commit or Saga, which is correct but slow or complex. Finally event sourcing with CQRS: a command validator appends events to a replicated log, a state machine applies them, and a read model serves queries."
3. Deep dive	25 min	"2PC is blocking and the coordinator is a single point of failure; Saga/TC-C is eventually consistent with compensation. Event sourcing avoids distributed transactions: events are append-only and ordered per partition, so I can reach high throughput with in-memory, local state and file-based logs (mmap). Reproducibility is the big win - replay from the start to prove any balance. For availability I replicate the log with Raft, and take periodic snapshots to speed up recovery. Transfers across partitions are modelled as an ordered pair of events and coordinated with a saga."
4. Wrap up	4 min	"Extras: snapshots and log compaction, idempotency, leader election tests, and monitoring replication lag. Clients can subscribe to updates rather than poll."

13. 60-second recap / practice checklist

- Evolution: sharded in-memory -> relational + 2PC/TC-C/Saga -> event sourcing with Raft (CQRS).
- Events are the source of truth; balances derived; replay gives reproducibility and audit.
- Partition accounts; cross-partition = TC/C or Saga with idempotent steps.

14. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Raft consensus
• Martin Fowler: Event sourcing
• Microservices.io: Saga pattern
• Martin Fowler: CQRS